TorchJD: Jacobian Descent in PyTorch

Valérian Rey — valerian.rey@gmail.com
Pierre Quinton — pierre.quinton@epfl.ch

April 13, 2026

GitHub
github.com/TorchJD/torchjd

TorchJD
torchjd.org

arXiv
arxiv.org/pdf/2406.16232

Motivation

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \quad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Partial order

Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} & \\ \Leftrightarrow \quad & u_i \leq v_i & \forall i \in [m] \end{aligned}$$

Examples:

- $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \npreceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Partial order

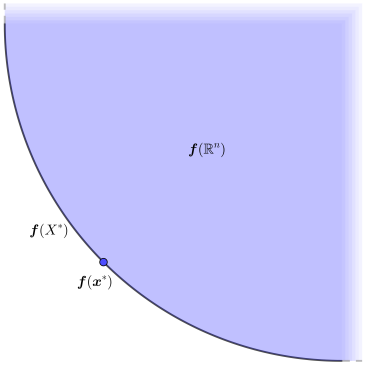
Define the partial order $(\mathbb{R}^m, \preceq)$, for $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m$, as

$$\begin{aligned} & \boldsymbol{u} \preceq \boldsymbol{v} \\ \Leftrightarrow \quad & u_i \leq v_i \qquad \forall i \in [m] \end{aligned}$$

Examples:

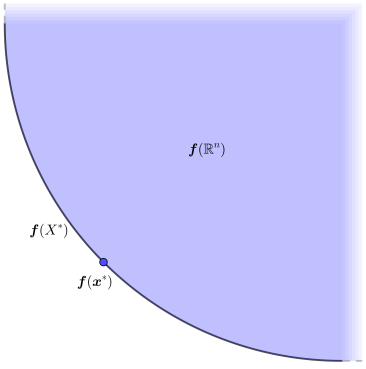
- $\begin{bmatrix} 2 \\ 1 \end{bmatrix} \preceq \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
- $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ are not comparable with this partial order.

Pareto optimality



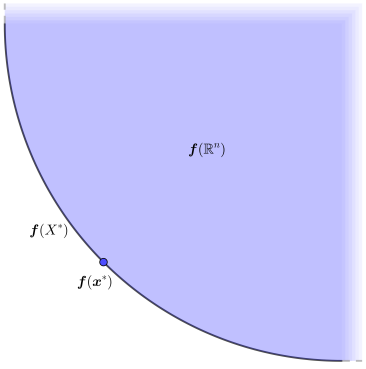
Pareto optimality

Pareto set: X^*



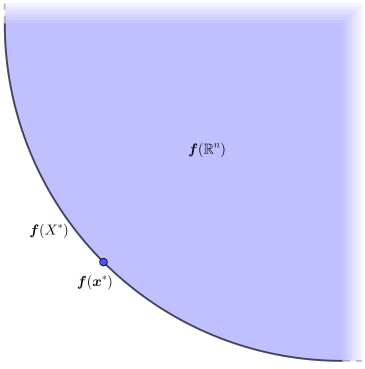
Pareto optimality

Pareto set: X^*
Pareto front: $\boldsymbol{f}(X^*)$



Pareto optimality

Pareto set: X^*
Pareto front: $\boldsymbol{f}(X^*)$



$$\boldsymbol{x}^* \in X^* \quad \Leftrightarrow \quad \forall \boldsymbol{y} \in \mathbb{R}^n,\ \boldsymbol{f}(\boldsymbol{y}) \preceq \boldsymbol{f}(\boldsymbol{x}^*) \rightarrow \boldsymbol{f}(\boldsymbol{y}) = \boldsymbol{f}(\boldsymbol{x}^*)$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order.

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y}$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y} \preceq \boldsymbol{f}(\boldsymbol{x})$$

Jacobian descent

Goal: minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ according to the $\preceq$ partial order. For any $\boldsymbol{x} \in \mathbb{R}^n$, the Jacobian of $\boldsymbol{f}$ at $\boldsymbol{x}$, is

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \nabla f_2(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_1(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\boldsymbol{x}) \\ \frac{\partial}{\partial x_1} f_2(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_2(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_2(\boldsymbol{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(\boldsymbol{x}) & \frac{\partial}{\partial x_2} f_m(\boldsymbol{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\boldsymbol{x}) \end{bmatrix}$$

For a small enough update $\boldsymbol{y} \in \mathbb{R}^n$, Taylor's theorem states

$$\boldsymbol{f}(\boldsymbol{x} + \boldsymbol{y}) \approx \boldsymbol{f}(\boldsymbol{x}) + \mathcal{J}\boldsymbol{f}(\boldsymbol{x}) \cdot \boldsymbol{y} \preceq \boldsymbol{f}(\boldsymbol{x})$$

We select $\boldsymbol{y} = -\eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x})) \in \mathbb{R}^n$.

Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

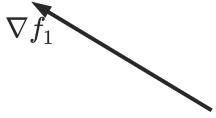
Jacobian descent

The mapping $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ is called an **aggregator**, it parameterizes **Jacobian descent**:

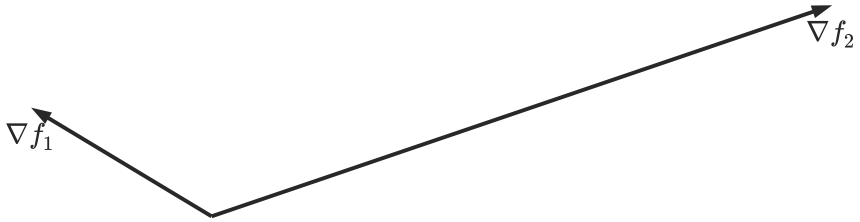
Algorithm 1: Jacobian descent with aggregator $\mathcal{A}$

Input: $\boldsymbol{x} \in \mathbb{R}^n$, $0 < \eta$, $T \in \mathbb{N}$, $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$
for $t \leftarrow 1$ **to** T **do**
  $\boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$
Output: $\boldsymbol{x}$

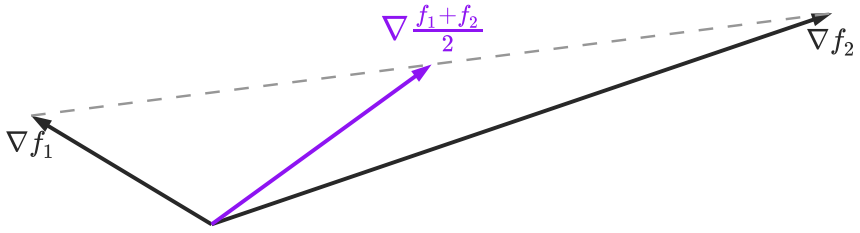
Jacobian descent



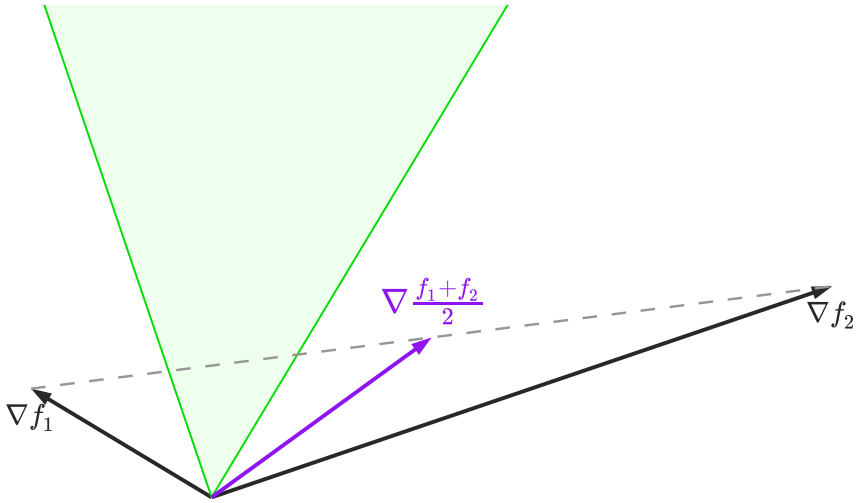
Jacobian descent



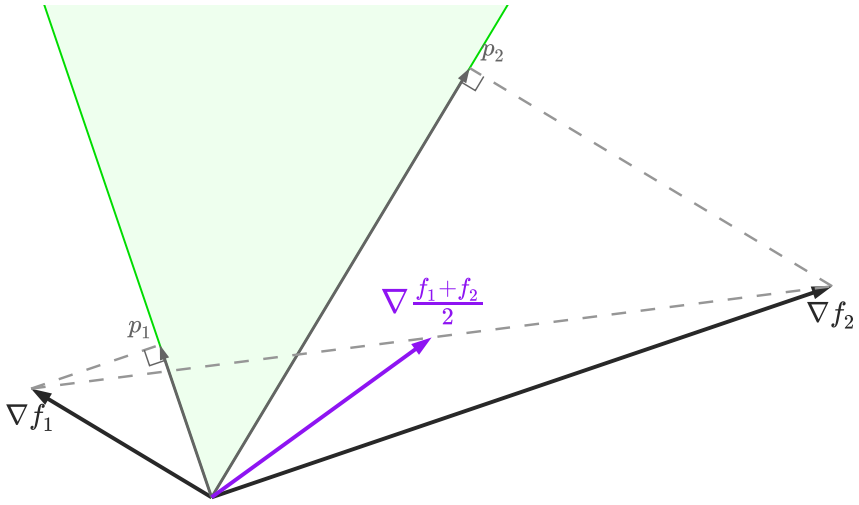
Jacobian descent



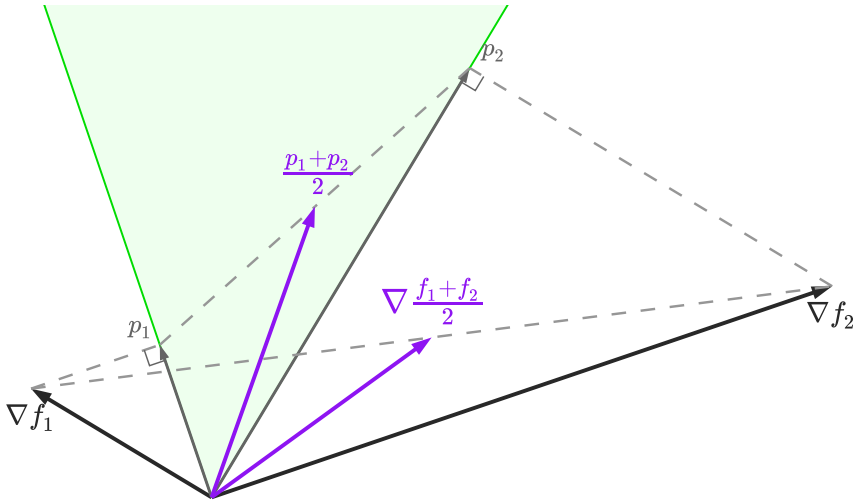
Jacobian descent



Jacobian descent



Jacobian descent

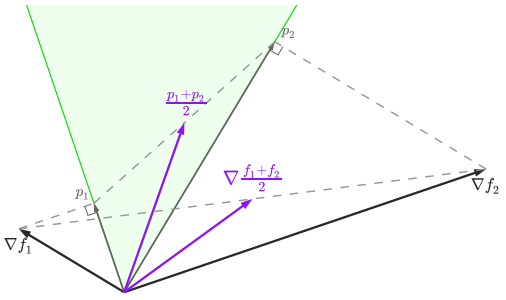


Formal definition of UPGrad

Given $J \in \mathbb{R}^{m \times n}$, let C_J be the cone $\{\boldsymbol{y} \in \mathbb{R}^n : \boldsymbol{0} \preceq J\boldsymbol{y}\}$.

$$p_i = \underset{\boldsymbol{y} \in C_J}{\arg\min} \|\nabla f_i - \boldsymbol{y}\|^2$$

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m} \sum_{i \in [m]} p_i$$



TorchJD

TorchJD: core functions

Library	Function	Description
`torch`	`autograd.grad`	compute gradients
`torch`	`autograd.backward`	compute gradients and accumulate in `.grad`

TorchJD: core functions

Library	Function	Description
`torch`	`autograd.grad`	compute gradients
`torch`	`autograd.backward`	compute gradients and accumulate in `.grad`
`torchjd`	`autojac.jac`	compute jacobians

TorchJD: core functions

Library	Function	Description
`torch`	`autograd.grad`	compute gradients
`torch`	`autograd.backward`	compute gradients and accumulate in `.grad`
`torchjd`	`autojac.jac`	compute jacobians
`torchjd`	`autojac.backward`	compute jacobians and accumulate in `.jac`

TorchJD: core functions

Library	Function	Description
`torch`	`autograd.grad`	compute gradients
`torch`	`autograd.backward`	compute gradients and accumulate in `.grad`
`torchjd`	`autojac.jac`	compute jacobians
`torchjd`	`autojac.backward`	compute jacobians and accumulate in `.jac`
`torchjd`	`autojac.jac_to_grad`	aggregate `.jac` fields into `.grad`

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss( reduction="none" )  # do not average over the batch
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss( reduction="none" )  # do not average over the batch

for input, target in dataloader:
  output = model(input)  # shape: [16]
  losses = loss_fn(output, target)  # shape: [16]
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss( reduction="none" )  # do not average over the batch

for input, target in dataloader:
  output = model(input)  # shape: [16]
  losses = loss_fn(output, target)  # shape: [16]
  autojac.backward(losses)
  # parameters now have a .jac field
```

TorchJD: basic usage

```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss(reduction="none")  # do not average over the batch

for input, target in dataloader:
  output = model(input)  # shape: [16]
  losses = loss_fn(output, target)  # shape: [16]
  autojac.backward(losses)
  # parameters now have a .jac field
  autojac.jac_to_grad(model.parameters(), UPGrad())
  # parameters now have a .grad field
```

TorchJD: basic usage

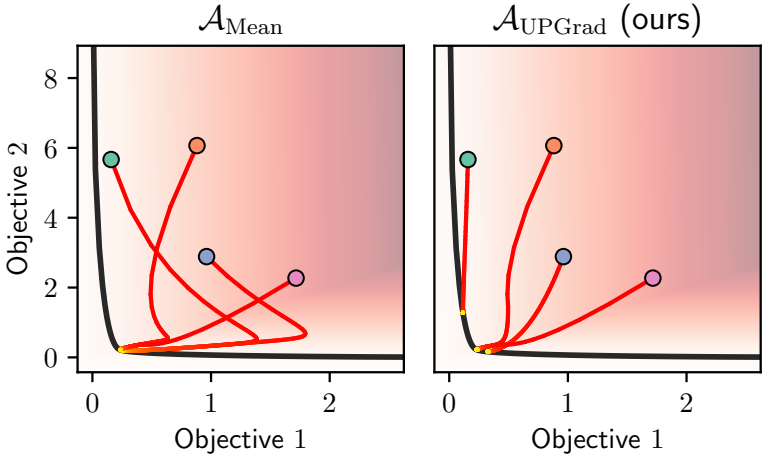
```
from torchjd import autojac
from torchjd.aggregation import UPGrad

batch_size = 16
loss_fn = MSELoss(reduction="none")  # do not average over the batch

for input, target in dataloader:
  output = model(input)  # shape: [16]
  losses = loss_fn(output, target)  # shape: [16]
  autojac.backward(losses)
  # parameters now have a .jac field
  autojac.jac_to_grad(model.parameters(), UPGrad())
  # parameters now have a .grad field
  optimizer.step()
  optimizer.zero_grad()
```

Optimization trajectories

Goal: Minimize some convex quadratic function $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$, with conflicting grads.

Optimization trajectories

Goal: Minimize some convex quadratic function $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$, with conflicting grads.



Discussion